IGVCore Architecture

IGVCore is a subset of the IGV.js codebase extracted into PGB. It provides two capabilities: loading and rendering gene annotations on a 2D canvas, and resolving gene names to genomic coordinates. PGB interacts with IGVCore through exactly two seams.

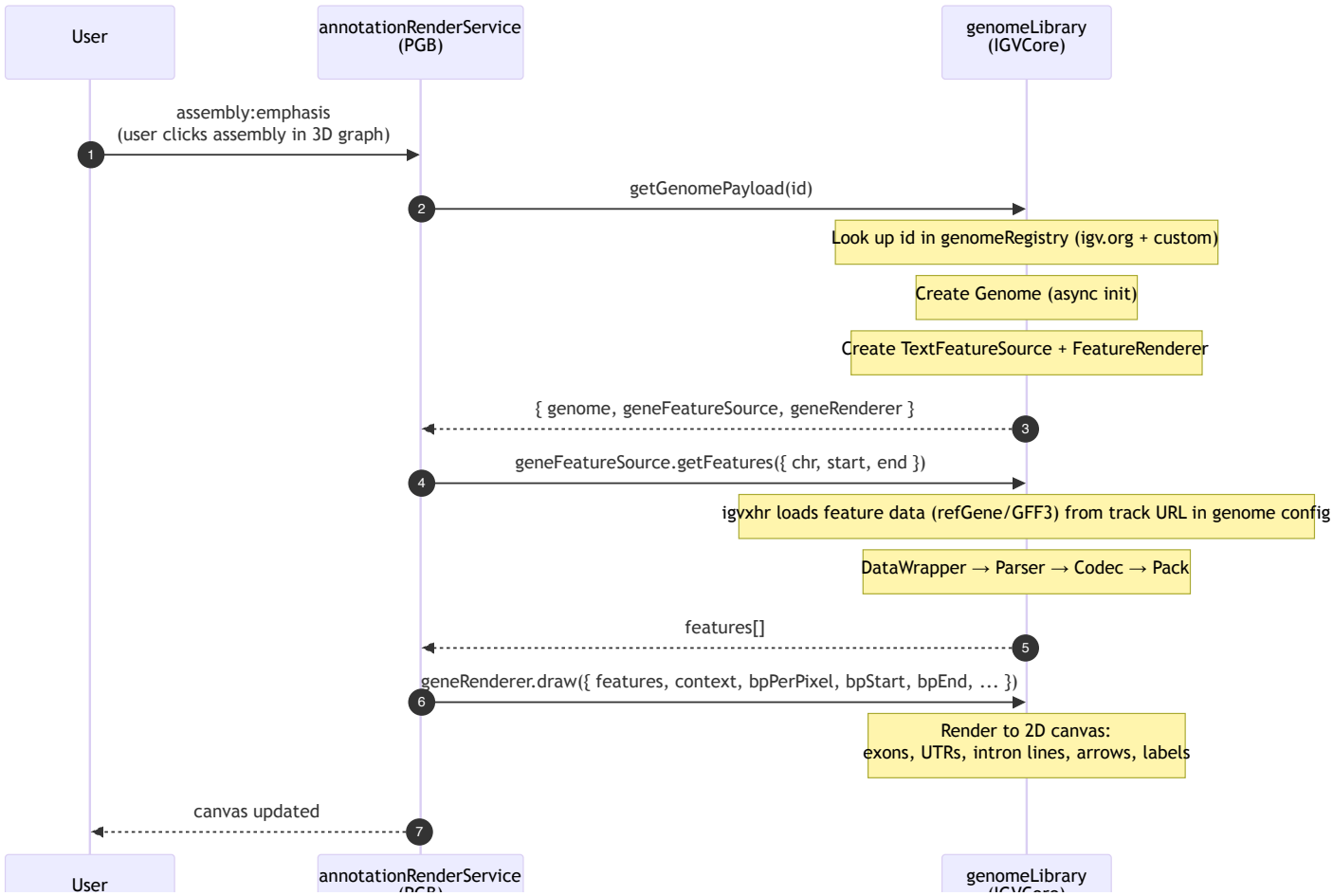
Directory Structure

```
src/igvCore/
├── genome/           Entry point: genomeLibrary.js
|   ├── genomeLibrary.js       Facade — creates genome payload from registry config
|   ├── genomeRegistry.js      Central registry — merges igv.org + custom sources
|   ├── igvOrgRegistrySource.js  Loads genome configs from igv.org/genomes/genomes3.json
|   ├── customRegistrySource.js   Loads genome configs from custom URL (set via setCustomRegistryURL)
|   ├── genomeIDAliases.js     Maps alternate genome IDs (e.g., hg38 → GRCh38)
|   ├── genome.js              Genome model (chromosomes, aliases, sequences)
|   ├── chromosome.js         Chromosome metadata
|   ├── chromSizes.js          Chromosome size loading
|   ├── chromAliasFile.js      Chromosome alias file parsing
|   ├── chromAliasDefaults.js  Default chromosome aliases
|   ├── chromAliasBB.js        BigBed chromosome alias loading
|   ├── cytoband.js            Cytoband model
|   ├── cytobandFile.js        Cytoband file parsing
|   ├── cytobandFileBB.js      BigBed cytoband loading
|   ├── loadSequence.js        Sequence loading orchestration
|   ├── indexedFasta.js        FASTA with .fai index
|   ├── nonIndexedFasta.js     FASTA without index
|   ├── cachedSequence.js      Sequence caching
|   ├── sequenceInterval.js    Sequence interval model
|   ├── genomicInterval.js    Genomic interval model
|   └── twobit.js              TwoBit sequence format
├── codec/            Pure parsing
|   ├── refGeneCodec.js        decodeGenePredExt, findUTRs, decodeExons (refGene format)
|   └── gff/                   GFF3 parsing
|       ├── gffCodec.js        decodeGFF3 — line-level GFF3 decoder
|       ├── gffHelper.js       Transcript assembly from GFF3 features
|       ├── gffFeature.js      GFFFeature, GFFTranscript classes
|       ├── parseAttributeString.js  GFF3 attribute parsing
|       └── so.js              Sequence ontology helpers (isTranscript, isExon, etc.)
├── io/               Data loading
|   ├── textFeatureSource.js   Loads + caches features, delegates to reader
|   ├── featureFileReader.js   Fetches raw data via igvxhr, delegates to parser
|   ├── dataWrapper.js         String/ByteArray line iterator
|   ├── baseFeatureSource.js   Abstract base with next/previous feature
|   ├── chromAliasManager.js   Maps chromosome names between sources
|   ├── binary.js              Binary parser for BigWig/TwoBit formats
|   ├── indexFactory.js        loadIndex — loads tabix index for indexed feature files
|   ├── tabixIndex.js          Tabix index parsing
|   ├── bgzBlockLoader.js      BGZip block loading for indexed files
|   ├── bgzLineReader.js       Line-by-line reader for BGZip-compressed files
|   └── bigwig/                BigBed/BigWig binary readers
|       ├── bwReader.js
|       ├── bwSource.js
|       ├── rpTree.js
|       ├── bpTree.js
|       ├── chromTree.js
|       ├── bbDecoders.js
```

```
|       └── trix.js
├── layout/           Feature packing
|   └── featurePacker.js       Row assignment for overlapping features
├── rendering/        Canvas drawing
|   ├── featureRenderer.js     Draws gene models (exons, UTRs, introns, arrows)
|   ├── featureRendererUtils.js  Coordinate conversion, translation dictionary
|   ├── exonUtils.js           Exon phase/start/end helpers
|   └── igvCanvas.js           Canvas drawing primitives (strokeLine, fillText, etc.)
├── search/           Gene name resolution
|   └── geneSearch.js          Calls IGV web service to resolve names to loci
├── feature/          Parsing glue
|   ├── featureParser.js       Configures decoder (refGene or GFF3), iterates lines → features
|   └── featureUtils.js       packFeatures (multi-chromosome wrapper)
├── util/             Shared utilities
|   ├── sequenceUtils.js       DNA complement/reverse-complement
|   ├── igvUtils.js            HTTP option building, data URL detection
|   ├── ucscUtils.js           AutoSQL parser (for BigBed schemas)
|   ├── colorPalletes.js       Color generation for canvas rendering
|   └── bufferUtils.js         Buffer/ArrayBuffer utilities
├── qtl/
|   └── qtlSelections.js       Null object for QTL phenotype selections
└── __tests__/        Characterization tests
    └── *.test.js              Tests for genomeLibrary, featureParser, codecs, etc.
```

Seam 1: Gene Annotations

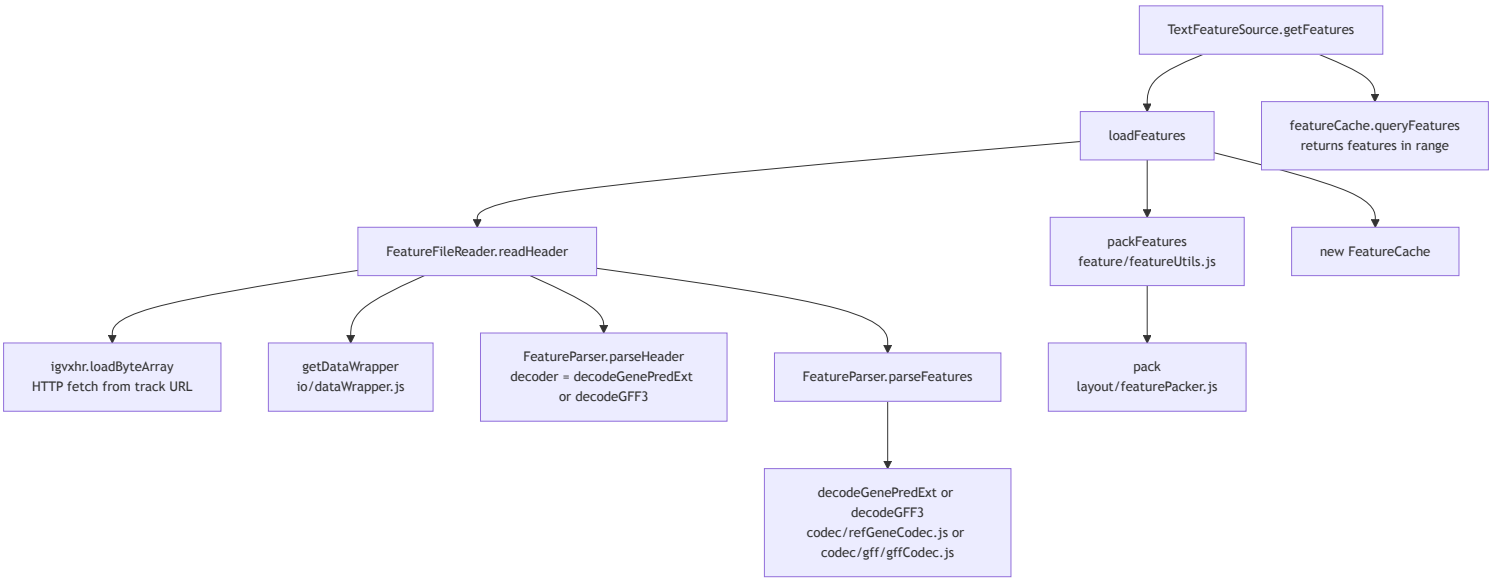
When a user emphasizes an assembly in the 3D graph, PGB renders gene annotations on a 2D canvas strip below the viewport.



(PGB) (IGVCore)

Data flow detail: loading features

When `textFeatureSource.getFeatures()` is called for the first time, the internal chain is:



For indexed feature files (e.g., GFF3.gz with .tbi), `FeatureFileReader` uses `indexFactory.loadIndex` and `BGZBlockLoader` / `BGZLineReader` to fetch only the blocks overlapping the requested range, rather than loading the entire file.

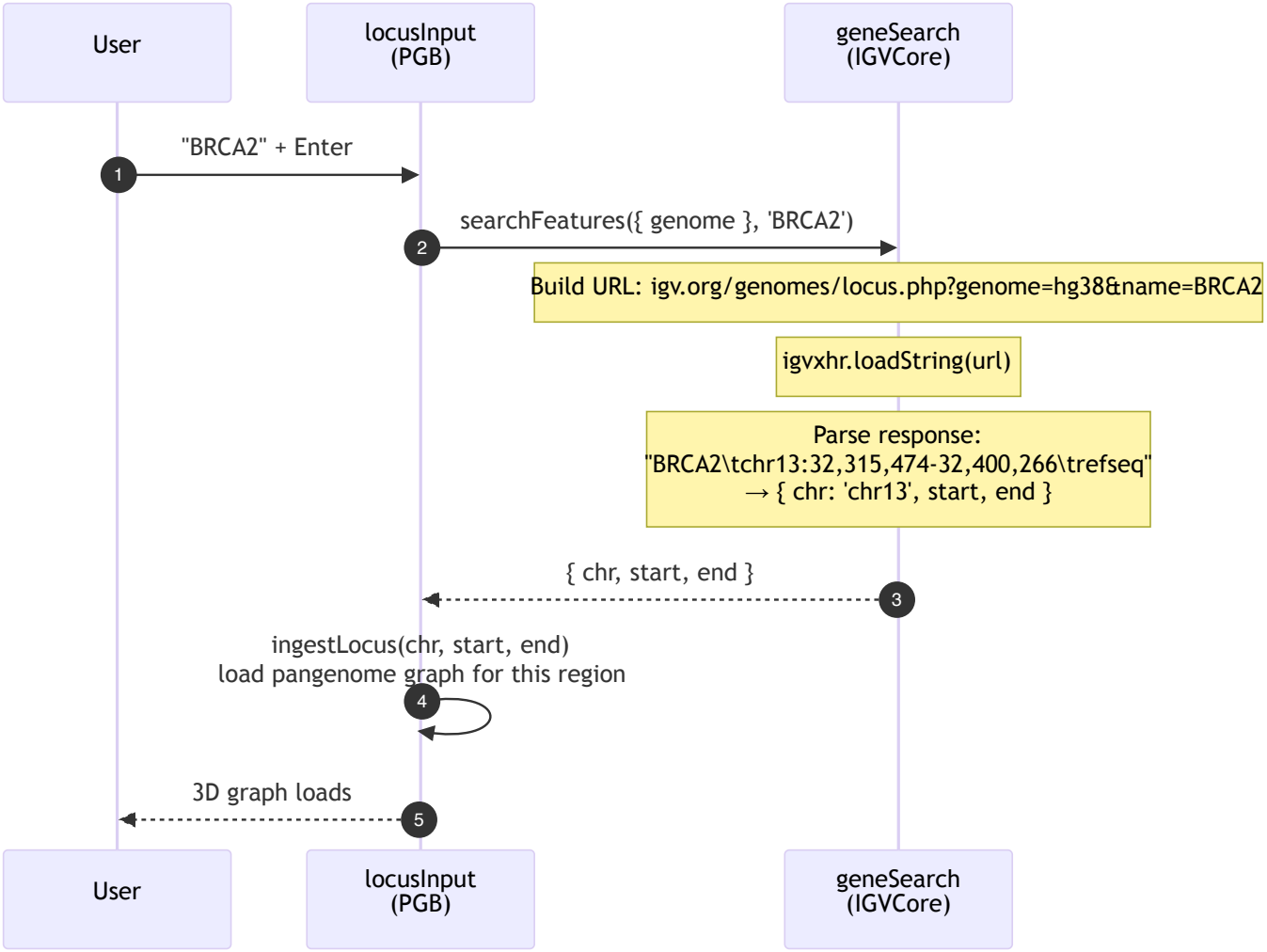
What the renderer draws

`FeatureRenderer.draw()` iterates features and for each one calls `render()`:

- **Single-exon features**: a filled rectangle with directional arrows (strand)
- **Multi-exon features**: a thin center line (intron) with thick rectangles (exons)
  - UTR exons are drawn at half-height
  - Coding exons at full height
  - Partial UTR/coding exons split at cdStart/cdEnd boundaries
  - Directional arrows along intron lines and within wide exons
- **Labels**: gene names centered below features (when zoom permits)
- **Amino acid rendering**: at very high zoom (< 0.25 bp/pixel), codons are translated and drawn over coding exons

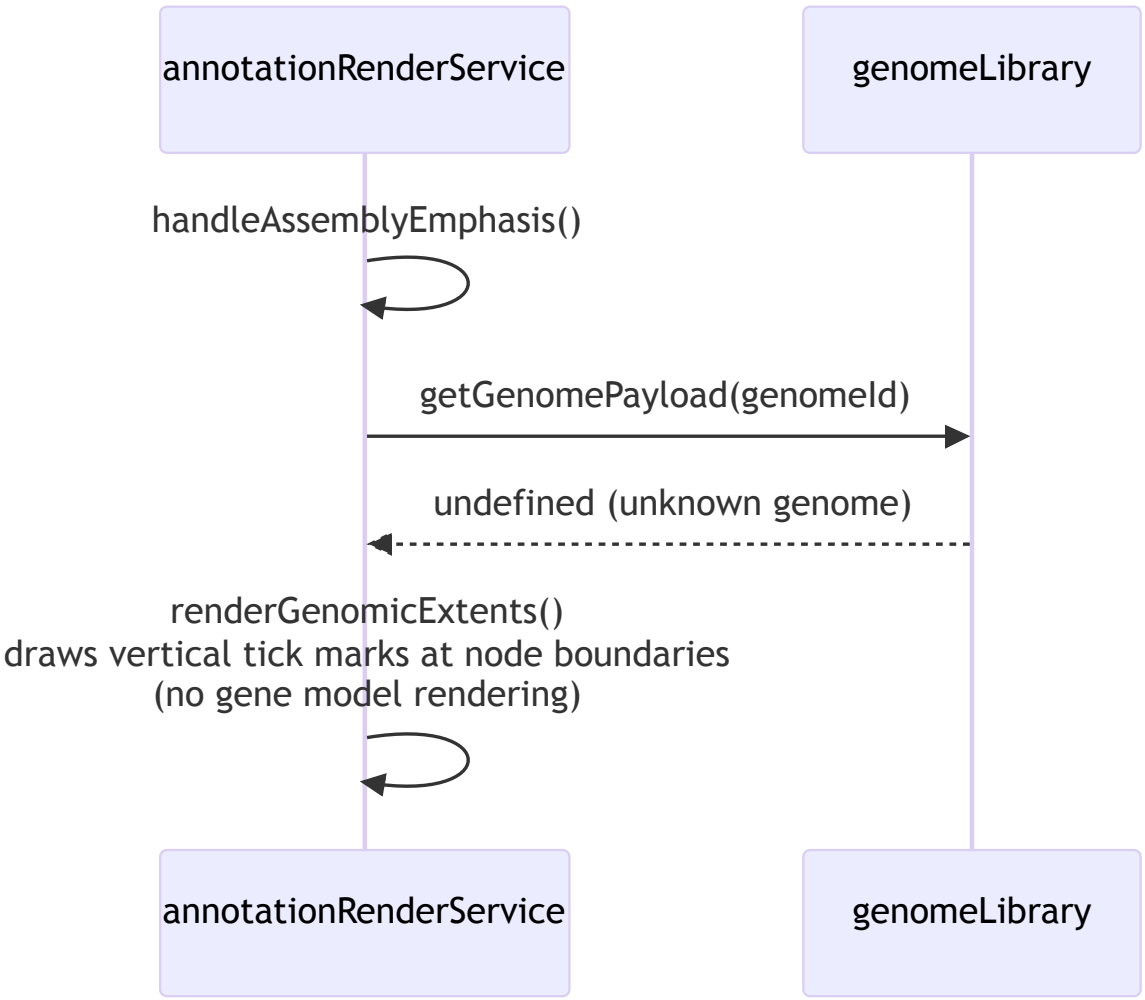
Seam 2: Gene Search

When a user types a gene name in the locus input, PGB resolves it to genomic coordinates via the IGV web service.



Unknown genome fallback

When an assembly's genome ID is not in the genome registry (e.g., a non-human reference or unregistered assembly), `getGenomePayload()` returns `undefined`. In this case, `annotationRenderService` falls back to rendering simple genomic extent markers (vertical tick marks at node boundaries) instead of gene annotations. This is the `hasGeneAnnotations = false` path.



What changed in the refactoring

The original IGVCore directory was a flat copy of IGV.js internals. The refactoring reorganized ~50 files into modules that reflect PGB's actual usage:

Before	After	What changed
`feature/decode/ucsc.js` (661 lines, 13 decoders)	`codec/refGeneCodec.js` (~100 lines, 1 decoder)	12 unused decoders deleted
`feature/` (14 files mixing I/O, parsing, rendering)	Split across `io/`, `rendering/`, `codec/`, `layout/`	Each module has a single concern

I/O, parsing, rendering)	`layout/`	
`feature/gff/` (4 files)	`codec/gff/` (gffCodec, gffHelper, gffFeature, parseAttributeString, so)	GFF3 parsing supported; used for format `gff3` / `gff` in track configs
Static `knownGenomes.js`	`genomeRegistry.js` + `igvOrgRegistrySource.js` + `customRegistrySource.js`	Genome configs loaded from igv.org and optional custom URL
`bigwig/` at top level	`io/bigwig/`	Binary I/O grouped with other I/O
`binary.js` at top level	`io/binary.js`	Same
`igv-canvas.js` at top level	`rendering/igvCanvas.js`	Grouped with rendering
`search.js` at top level	`search/geneSearch.js`	Own module
No tests	22 characterization test files	Feathers-style safety net for future changes

PGB entry points

- `src/main.js` → `import GenomeLibrary from "./igvCore/genome/genomeLibrary.js"`
- `src/main.js` → `import { initializeGenomeRegistry, setCustomRegistryURL } from "./igvCore/genome/genomeRegistry.js"` — registry must be initialized before `getGenomePayload`
- `src/locusInput.js` → `import { searchFeatures } from "./igvCore/search/geneSearch.js"`